Predicting Median Housing Values in California:
A Statistical and Machine Learning Analysis

Technical Report

Prepared by: Francois Lubin & Angelica Johnson
Final Project: Statistical Analysis & Predictive Modeling Team Project
Date: October 19, 2026

Executive Summary

This report documents the end-to-end statistical analysis of the California Housing dataset (1990 U.S. Census, 20,640 block-group records), covering data cleaning, exploratory data analysis (EDA), predictive model development, and validation. After cleaning, 18,575 records remained for analysis. Two candidate models were compared to predict median house value: a multiple linear regression baseline and a random forest regressor. Using 5-fold cross-validation and a held-out 20% test set, we selected the random forest regressor as the final model, which achieved a test RMSE of approximately \$44,715 and $R^2 = 0.796$, compared with \$58,464 RMSE and $R^2 = 0.652$ for linear regression. Median household income was the most influential predictor in the random forest (by feature and permutation importance) and the largest numeric coefficient in the linear model. Residual diagnostics revealed right-skewed, mildly heteroscedastic residuals (Shapiro-Wilk $p < 0.001$), a limitation discussed in Sections 7.2 and 8. Additional robustness checks (Sections 7.4–7.6) show that the forest outperforms linear regression in every cross-validation fold, that errors are larger for high-value districts, and that accuracy for geographically new regions is about 35% worse than the random-split estimate.

Table of Contents

- 1. Introduction
- 2. Data Description
- 3. Data Cleaning and Preparation
- 4. Exploratory Data Analysis
- 5. Model Selection and Methodology
- 6. Model Analysis: Results
- 7. Model Analysis: Statistical Validity and Model Selection Discussion
- 8. Limitations
- 9. Conclusion and Recommendations
- 10. References
- 11. Appendix: Code Output

1. Introduction

Housing affordability and valuation are central questions in applied statistics and urban economics. This project analyzes the California Housing dataset, derived from the 1990 U.S. Census, to build a model that predicts the median house value of a census block group from geographic, demographic, and housing-stock characteristics. The objective is twofold: (1) to produce a statistically sound, well-validated predictive model, and (2) to communicate the reasoning behind model selection using appropriate statistical diagnostics rather than accuracy metrics alone.

The project follows a standard applied statistics workflow: data cleaning and preparation, exploratory data analysis, model development and comparison, and validation of the final model's statistical assumptions.

2. Data Description

The raw dataset contains 20,640 observations across 10 variables: longitude, latitude, housing median age, total rooms, total bedrooms, population, households, median income, median house value (target), and ocean proximity (categorical: <1H OCEAN, INLAND, NEAR OCEAN, NEAR BAY, ISLAND). Each row represents a census block group rather than an individual household.

Variable	Description
Median house value	Target variable: median value of owner-occupied homes (USD)
Median income	Median household income (tens of thousands USD)
Housing median age	Median age of houses in the block group (years)
Total rooms / total bedrooms	Aggregate room and bedroom counts for the block group
population / households	Aggregate population and household counts
Ocean proximity	Categorical proximity to the ocean

3. Data Cleaning and Preparation

Cleaning followed a reproducible pipeline (src/data_prep.py) applying the following steps in order:

- Duplicate removal: exact duplicate rows were dropped (0 found).
- Invalid record removal: rows with non-positive housing median age, total rooms, households, or median house value were removed, along with rows where total bedrooms logically exceeded total rooms.
- Top-coded target handling: median house value is capped at $500,001 in the source data, producing an artificial spike at that value. We removed the 965 top-coded records; the model's scope is therefore restricted to block groups with median values below $500,001.
- Missing-value imputation: total bedrooms had 207 missing values (1.0% of records). These were imputed using the median total bedrooms within the same ocean proximity group, a context-aware alternative to a single global median. Imputed values were capped at that row's total rooms to preserve logical consistency.
- Feature engineering: three ratio features were derived — rooms per household, bedrooms per room, and population per household — which are more directly informative than the raw aggregate counts.
- Outlier removal: the 1.5×IQR rule was applied to rooms per household and population per household, removing 1,100 extreme records (e.g., block groups with implausibly high rooms-per-

household ratios). This step was applied to the full dataset before the train/test split, so the test set is also filtered; Section 7.6 quantifies the effect.

After cleaning, 18,575 of the original 20,640 records (90.0%) remained for analysis.

4. Exploratory Data Analysis

4.1 Distribution of the Target Variable

The distribution of median house value is right-skewed (skewness = 0.78), with a mean of $193,935 and median of $176,400. This is consistent with typical income and price distributions, where a majority of block groups cluster at moderate values while a smaller number of higher-value areas extend the right tail.

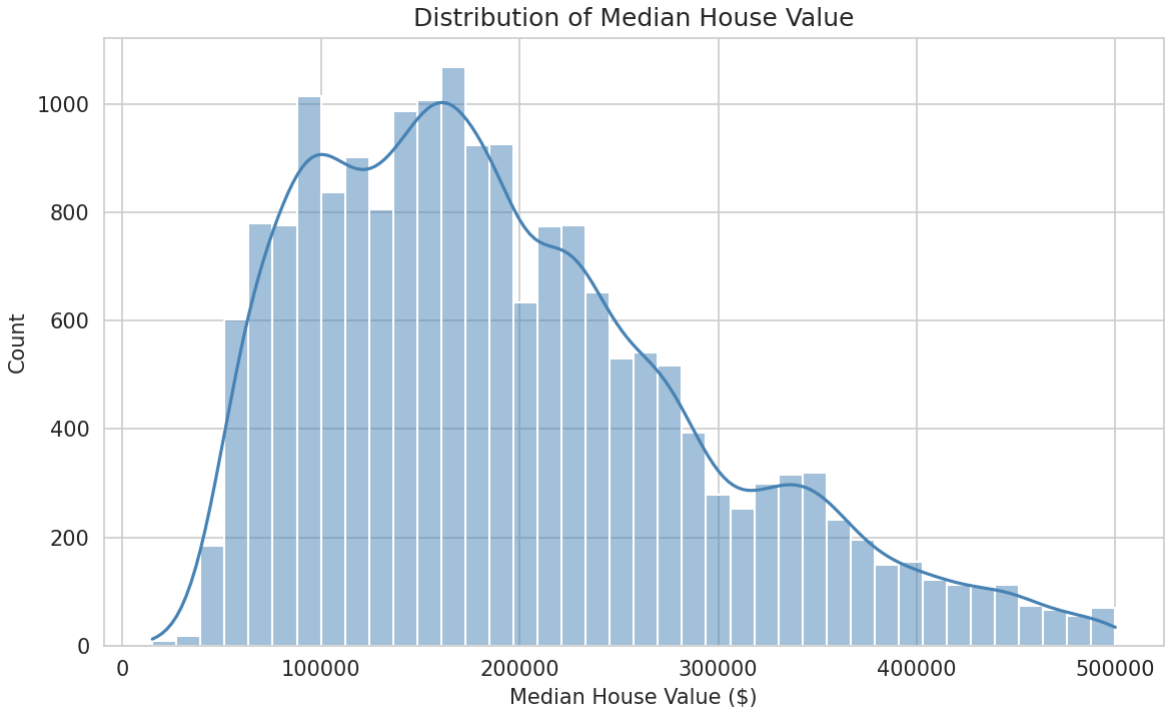


Figure 1. Distribution of median house value after cleaning.

4.2 Correlation Structure

Median income shows the strongest linear association with the target (r = 0.64), consistent with economic expectations. Rooms per household and bedrooms per room are strongly negatively correlated (r = -0.80), reflecting that block groups with more rooms per household tend to have proportionally fewer bedrooms per room (larger, non-bedroom living space).

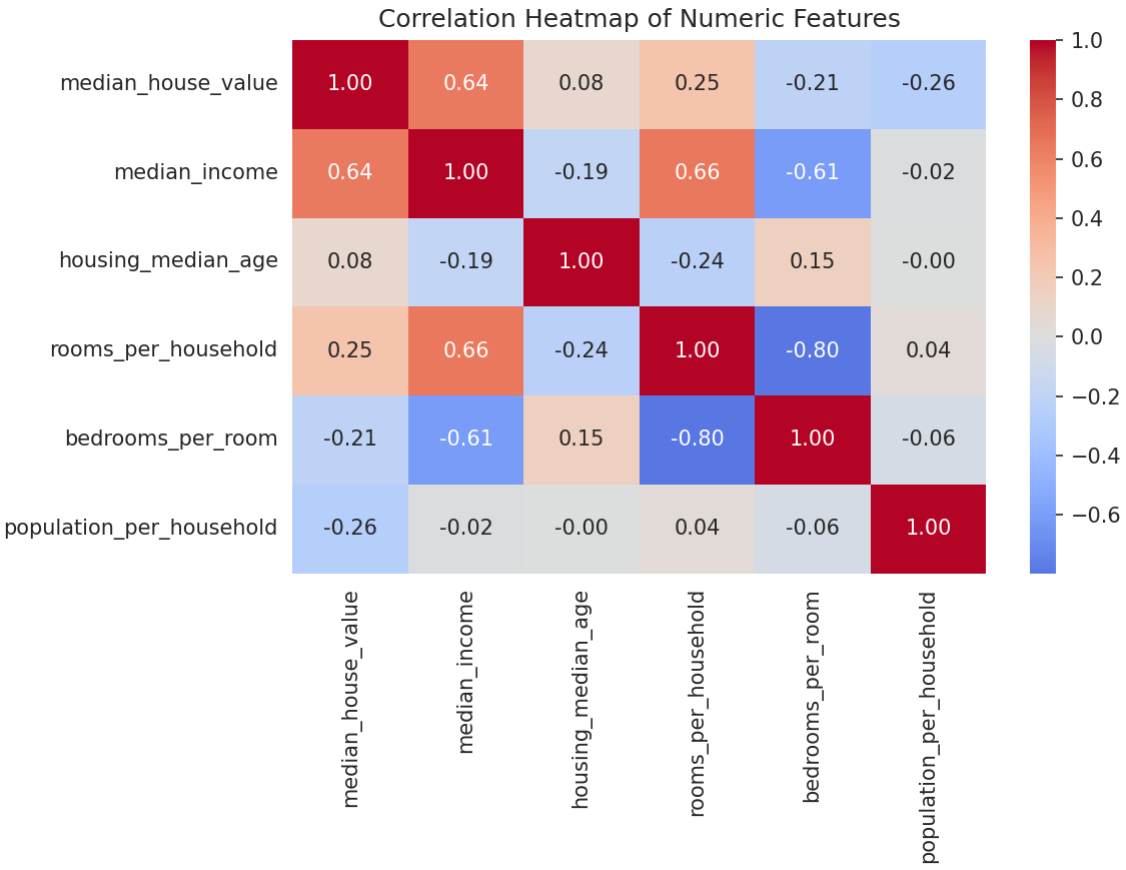


Figure 2. Correlation heatmap of numeric features.

4.3 Income vs. House Value



Figure 3. Median income vs. median house value. The positive association is clearly visible, along with increasing spread (heteroscedasticity) at higher income levels.

Figure 3 shows a clear positive, roughly linear relationship between income and house value ($r = 0.64$). The vertical spread of points widens as income rises, so equally wealthy neighborhoods can have quite different home values; this is the source of the mild heteroscedasticity examined in Section 7.2.

4.4 Geographic Distribution

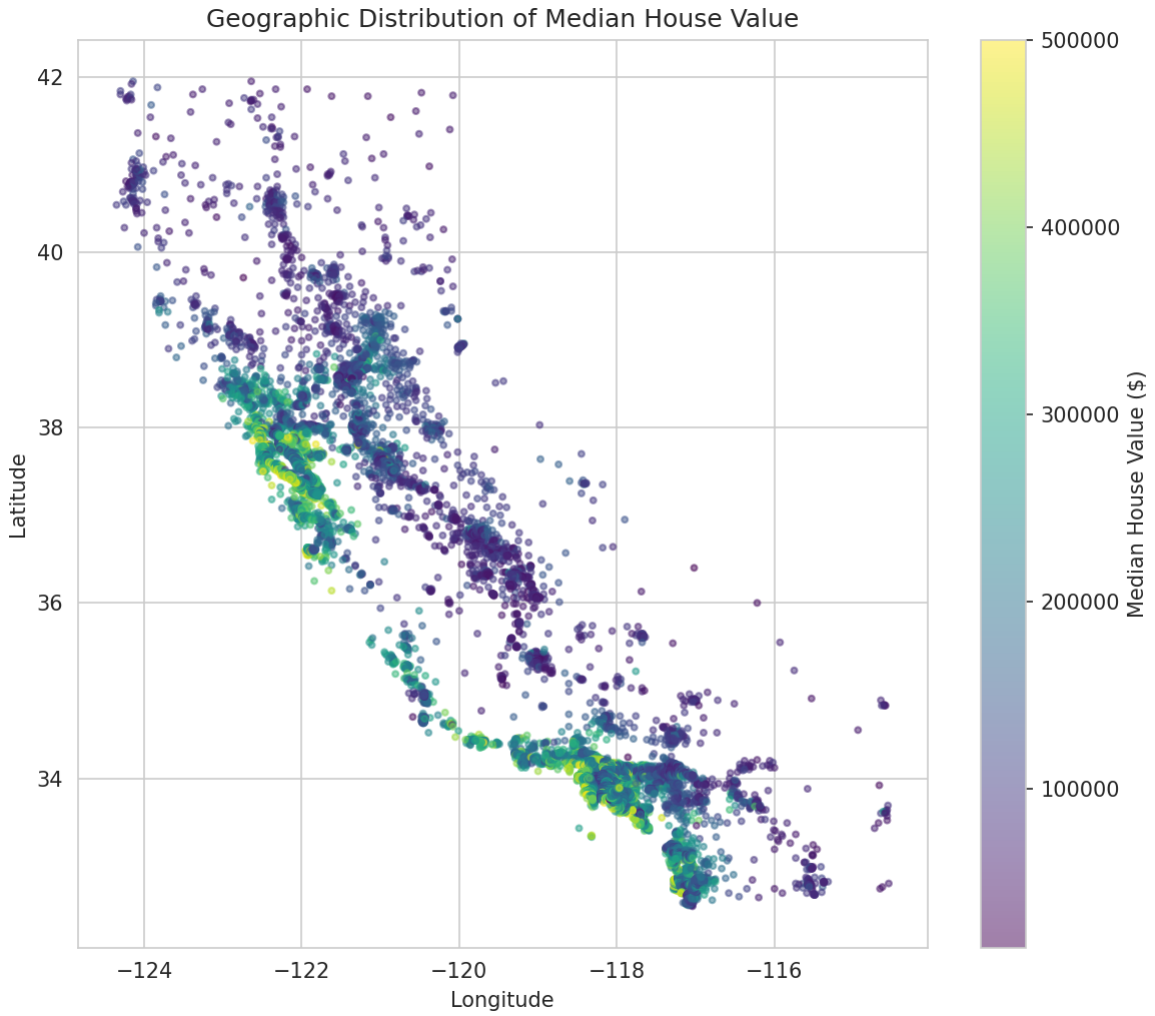


Figure 4. Geographic distribution of median house value across California. Coastal areas, particularly near the Bay Area and Los Angeles, show visibly higher values.

Figure 4 maps value by coordinates. The highest values cluster along the coast, particularly around the San Francisco Bay Area and the Los Angeles–San Diego corridor, while the interior Central Valley is cheaper. This is why longitude and latitude carry predictive information beyond income alone (Section 6.1).

4.5 House Value by Ocean Proximity

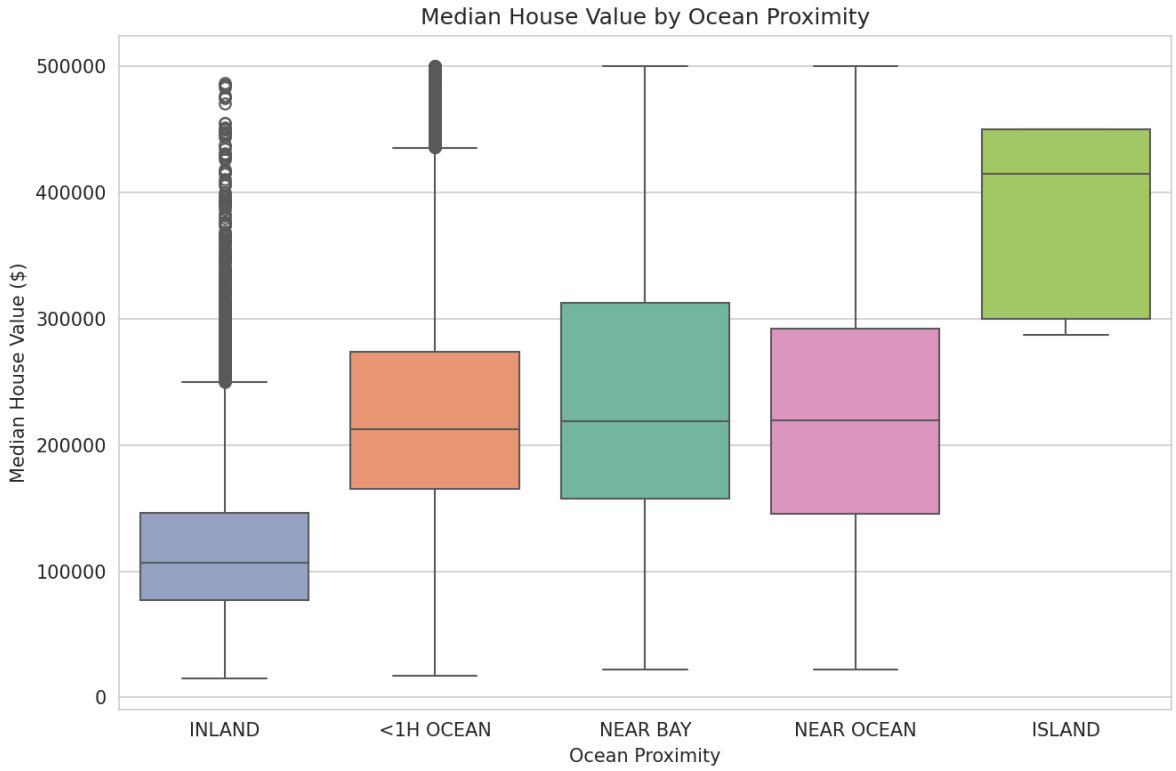


Figure 5. Median house value by ocean proximity category. INLAND block groups show systematically and substantially lower values, later confirmed as the second most important predictor in the random forest model.

Mean house value is $121,901 for INLAND block groups (n = 6,063), compared with $226,818 for <1H OCEAN, $228,678 for NEAR OCEAN, and $236,579 for NEAR BAY. The ISLAND category contains only 5 records (mean $380,440), too few to estimate reliably; see Section 7.3.

5. Model Selection and Methodology

The cleaned data were split into training (80%, n = 14,860) and test (20%, n = 3,715) sets using a fixed random seed for reproducibility. All numeric features were median-imputed (safeguarding against any residual missingness) and standardized; the categorical ocean proximity feature was one-hot encoded. Both steps were encapsulated in a scikit-learn column Transformer so that imputation medians, scaling parameters, and category lists are learned from the training data only. The cleaning steps in Section 3 (outlier removal and top-code exclusion), however, were applied to the full dataset before the split; their effect is quantified in Section 7.6.

Two models were trained and compared:

- Linear Regression: an ordinary least squares baseline, included to establish an interpretable reference point and test whether a linear, additive model adequately captures the relationship between predictors and price.
- Random Forest Regressor (Breiman, 2001): an ensemble of 150 regression trees (max depth 18, min samples leaf = 3), selected as a candidate capable of capturing non-linear relationships and interaction effects (e.g., the value of income may depend on geographic location) without requiring explicit interaction terms. Tree depth and count were chosen by hand to keep the saved model small and were not tuned by search. A comparison with a 300-tree unconstrained forest (src/hyperparam_check.py) gave nearly identical test accuracy (RMSE $44,520, R^2 0.798 versus $44,715 and 0.796) but a saved model about eight times larger (about 402 MB versus 52 MB uncompressed).

Model comparison used 5-fold cross-validation on the training set, scored by root mean squared error (RMSE), followed by a final, single evaluation of each fitted model on the held-out test set to obtain an estimate of performance on data not used for fitting. The model with the lower mean cross-validated RMSE was selected as final and re-evaluated on the test set for reporting.

6. Model Analysis: Results

Model	**CV RMSE (mean ± SD)**	**Test RMSE**	**Test MAE**	**Test R^2**
Linear Regression	$57,478 ± $1,208	$58,464	$43,137	0.652
Random Forest (final)	**$45,204 ± $1,391**	**$44,715**	**$29,899**	**0.796**

Note: ± is the sample standard deviation (n − 1) of RMSE across the five cross-validation folds.

The random forest reduced test RMSE by approximately 24% relative to linear regression and improved R^2 from 0.652 to 0.796, indicating it explains a substantially larger share of the variance in median house value. Training and test performance differ by model. Linear regression scores almost identically on both (R^2 0.653 vs. 0.652; RMSE $57,433 vs. $58,464), so it is not overfit but is limited by its linear form. The random forest fits the training data much more closely (R^2 0.935, RMSE $24,832) than the test data (R^2 0.796, RMSE $44,715), as random forests typically do. What matters for generalization is that held-out error is stable: cross-validated and test RMSE differ by less than 2% for both models, and the forest remains clearly better on data it has not seen.

6.1 Feature Importance (Random Forest)

Feature	Importance
median income	0.429
ocean proximity: INLAND	0.181
Population per household	0.111
longitude	0.077
latitude	0.069

Median income alone accounts for roughly 43% of the random forest's total feature importance, consistent with its strong marginal correlation identified in the EDA. Notably, longitude and latitude together contribute meaningfully (about 15% combined), confirming that geographic location carries predictive information beyond what is captured by ocean proximity alone — consistent with the visible coastal-value gradient in Figure 4.

Permutation importance on the test set, which measures how much RMSE worsens when one feature is shuffled, gives a similar ranking: median_income ($42,430), latitude ($20,576), ocean_proximity ($19,897), longitude ($19,357), and population_per_household ($17,601) matter most, while housing_median_age adds $6,562 and every other feature less than $1,200. Because impurity-based importances can be inflated for correlated features, this check supports the conclusion that income and location drive the predictions.

Linear-model coefficients should be read with care because several predictors are strongly collinear. Variance inflation factors are 92.9 for households, 65.9 for total_bedrooms, 26.1 for population, and 23.9 for total_rooms, with latitude and longitude near 10. Predictions are not affected, but individual coefficients for these variables are unstable. The most reliable coefficient is median_income: a one-standard-deviation increase is associated with about +$60,845 in predicted value, holding other variables fixed. The ISLAND coefficient (+$122,803) rests on only 5 records, none of which fall in the test set, and should not be interpreted.

7. Model Analysis: Statistical Validity and Model Selection Discussion

7.1 Why Random Forest Was Selected

Model selection was based on cross-validated RMSE rather than a single train/test split, reducing the risk that the comparison reflects one particular partition of the data (an application of the sampling-variability principle: a single point estimate of performance carries sampling error, while averaging over 5 folds gives a more stable estimate of true generalization error). The random forest's lower cross-validated RMSE (by about $12,274, or 21%) is consistent across folds; the fold-to-fold RMSE standard deviation is $1,391 for the forest and $1,208 for linear regression.

Because both models were evaluated on the same five folds, the comparison can be made pairwise. The forest had lower RMSE than linear regression in all five folds, by between $11,632 and $13,155 (mean difference $12,274, SD $627; 95% confidence interval $11,496 to $13,053). With only five folds, this interval is approximate because folds share training data, but the margin is much larger than the fold-to-fold variation.

For the final model, a bootstrap over the test set (resampling the 3,715 predictions) gives 95% confidence intervals of $42,753–$46,809 for RMSE, 0.778–0.812 for R^2, and $28,821–$31,019 for MAE. These intervals describe uncertainty from the finite test sample only; they do not capture the data-cleaning and spatial-generalization effects examined in Sections 7.5 and 7.6.

The magnitude of improvement also matches the underlying data structure observed in EDA: the relationship between predictors (especially geographic coordinates and ocean proximity interacting with income) and house value is not purely linear or additive. Random forests can model such interactions and non-linearities without explicit specification, which is a plausible statistical explanation for its improved fit over ordinary least squares.

7.2 Residual Diagnostics

For a regression model to support valid inference (beyond point prediction), residuals should be approximately normally distributed with constant variance (homoscedasticity). These assumptions were assessed for the final model on the test set:

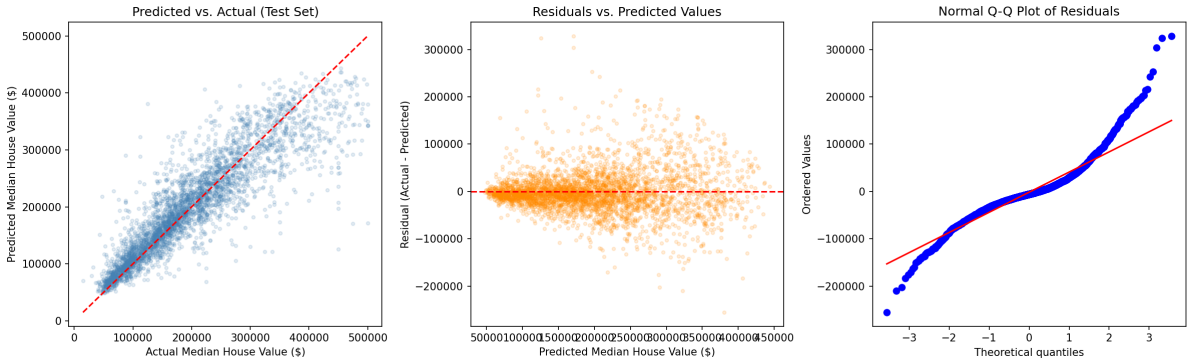


Figure 6. Predicted vs. actual values (left), residuals vs. predicted values (center), and a normal Q-Q plot of residuals (right) for the final random forest model on the test set.

- Predicted vs. actual (left panel): points cluster around the 45° reference line, confirming the model tracks the target reasonably well across most of its range, with somewhat more scatter at higher home values.

- Residuals vs. predicted (center panel): residual spread increases modestly at higher predicted values, indicating mild heteroscedasticity — the model is less precise for higher-value block groups than for lower-value ones.
- Normal Q-Q plot (right panel): residuals deviate from the reference line in both tails, most visibly in the upper tail, indicating the residual distribution is right-skewed with heavier tails than a normal distribution.

A Shapiro-Wilk test (Shapiro & Wilk, 1965) on all 3,715 test-set residuals formally rejected the null hypothesis of normality ($W = 0.908$, $p < 0.001$). Given the very large sample size, even small departures from normality can produce statistically significant results; the Q-Q plot provides more informative, practically interpretable evidence of the skew's magnitude.

7.3 Implication for Model Use

The residual non-normality does not invalidate the model's point predictions — random forests make no distributional assumption about residuals during training, unlike ordinary least squares. However, it does mean that naive symmetric prediction intervals (e.g., prediction ± 1.96 × RMSE) would understate uncertainty for high-value block groups and should not be used for formal interval estimation without adjustment (e.g., via quantile regression forests or bootstrap-based asymmetric intervals).

7.4 Where the Model Is Less Accurate

Errors are not uniform. Grouped by actual house value, the model over-predicts cheap districts and under-predicts expensive ones, a typical regression-to-the-mean pattern. Error is largest for the most expensive districts, which also explains the right-skewed residuals in Section 7.2.

Actual value	n	RMSE	MAE	Mean pred − actual
≤ $100k	702	$28,077	$17,926	+$15,287
$100–200k	1,484	$34,988	$23,896	+$13,994
$200–300k	967	$42,086	$31,622	−$325
$300–400k	407	$56,191	$42,753	−$25,724
$400–500k	155	$115,286	$97,102	−$96,630

By region, errors are lowest for INLAND block groups (RMSE $34,322) and highest for NEAR OCEAN ($56,145) and NEAR BAY ($52,813), where prices are higher and more variable. The ISLAND category has no test records.

Region	n	RMSE	MAE	Mean pred − actual
<1H OCEAN	1,622	$45,384	$31,858	+$3,142
INLAND	1,195	$34,322	$21,200	+$1,903
NEAR BAY	417	$52,813	$37,184	+$1,968
NEAR OCEAN	481	$56,145	$38,592	−$5,101

7.5 Generalization to New Regions

The random train/test split lets the model see neighboring block groups during training, which flatters accuracy when predicting for a place it has never seen. To test this, block groups were clustered geographically with k-means and whole clusters were held out in cross-validation. Spatial-block RMSE was $59,977 ± $7,806, compared with $44,347 ± $1,393 for random folds under the same procedure, about 35% worse. The headline figures in Section 6 should therefore be read as accuracy for areas similar to those in the training data, not for new regions.

7.6 Sensitivity to Cleaning Choices

Cleaning (Section 3) was applied before the split, so we re-ran the comparison under alternative choices. The random forest beats linear regression in every variant; only the absolute level of error changes.

Variant	RF RMSE	RF R^2	Linear RMSE	Linear R^2
Team pipeline	$44,715	0.796	$58,464	0.652
No IQR outlier removal	$46,363	0.784	$62,105	0.613
Split first, test keeps outliers	$46,389	0.784	$62,177	0.612
Split first, test filtered	$46,332	0.785	$59,521	0.645
Keep $500,001 rows, all test rows	$49,501	0.819	$64,901	0.688
Keep $500,001 rows, scored on < $500k	$44,990	n/a	$57,385	n/a

Outlier filtering of the test set improves reported RMSE by about $1,600 (3.6%) for the forest, so the headline figure is somewhat optimistic for unfiltered data. Keeping the capped $500,001 rows raises R^2 only because the target range widens; on the rows below the cap, accuracy is essentially unchanged.

8. Limitations

- Top-coding exclusion: removing records at the $500,001 cap means the model does not extrapolate reliably to California's highest-value housing markets.
- Data age: the dataset reflects 1990 Census conditions; relationships between income, location, and housing value may have shifted substantially in the intervening decades, so the model should not be used for current market prediction without re-fitting on recent data.
- Ecological-level data: each record aggregates an entire census block group, not an individual household; inferences should not be applied to individual homes (an instance of the ecological fallacy).
- Mild heteroscedasticity and larger errors for expensive districts: prediction uncertainty is not constant across the price range, as shown in Section 7.2, and should be communicated accordingly rather than as a single fixed margin of error.
- Interpretability trade-off: the random forest, while more accurate, is less directly interpretable than linear regression; feature importances describe overall influence but not the sign or functional form of each effect.
- Cleaning scope: IQR outlier removal narrows the population the model applies to and flatters test metrics by roughly $1,600 (3.6%) relative to unfiltered data (Section 7.6).

- Spatial generalization: a random split overstates accuracy for new regions; spatial-block cross-validation gives RMSE about 35% higher (Section 7.5).
- Sparse category: ISLAND has only 5 records, so its effect cannot be estimated reliably.
- Limited tuning: hyperparameters were set by hand, and only two model families were compared; gradient boosting or tuned models could perform better.

9. Conclusion and Recommendations

This analysis cleaned, explored, and modeled the California Housing dataset, comparing a linear regression baseline against a random forest regressor using cross-validated statistical evaluation rather than a single accuracy figure. We selected the random forest as the final model because it consistently reduced RMSE and improved R^2 in cross-validation, supported by EDA showing that income and location interact in ways a purely linear model cannot capture. Residual diagnostics confirmed reasonable point-prediction accuracy while identifying right-skewed, mildly heteroscedastic residuals that should inform how prediction uncertainty is communicated. Robustness checks show the forest's advantage holds in every fold and under alternative cleaning choices, but that accuracy is lower for high-value districts and for geographically new regions, so the reported figures should be read as an estimate for areas similar to the training data.

Recommendations

- Use the random forest for point predictions of median house value in areas similar to the training data, and report them with an error of about ±$45,000 (RMSE); expect larger errors, about $56,000 to $115,000, for districts valued above $300,000 (Section 7.4).
- Do not apply the model to new regions without retraining or re-validating it on local data; spatial cross-validation raised RMSE by about 35% (Section 7.5).
- Do not interpret individual coefficients of the linear model for rooms, bedrooms, population, or households, because they are strongly collinear (Section 6.1); if interpretation is needed, use regularized regression or principal components.
- Before any production use, update the data (the source is the 1990 Census) and consider keeping the capped $500,001 districts with an appropriate censored-data method, since removing them narrows the model's scope (Section 8).
- Next steps for accuracy: tune hyperparameters with nested cross-validation, compare gradient boosting, and use quantile regression forests or bootstrap methods for asymmetric prediction intervals (Section 7.3).

10. References

Pace, R. K., & Barry, R. (1997). Sparse spatial autoregressions. Statistics & Probability Letters, 33(3), 291–297. (Original source of the California Housing dataset.)

Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32.

Géron, A. (2019). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd ed.). O'Reilly Media. (Dataset distribution used in this project.)

Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Shapiro, S. S., & Wilk, M. B. (1965). An analysis of variance test for normality (complete samples). Biometrika, 52(3–4), 591–611.

11. Appendix: Code Output

This appendix reproduces the console output of the project scripts (src/data_prep.py, src/model.py, src/rigor.py, src/sensitivity.py, and src/hyperparam_check.py) run in order on the raw data, as in Running-full-pipeline.ipynb. Output is shown unedited except that local file paths are replaced by /Users/francoislubin/Downloads/California-housing-project/data/housing.csv. The numbers match those in Sections 3 to 7.

Output of: python3 src/data_prep.py

```
"""Data cleaning and feature engineering for the California Housing project.…

import logging
from pathlib import Path

import numpy as np
import pandas as pd

logging.basicConfig(level=logging.INFO, format="%(levelname)s: %(message)s")
logger = logging.getLogger(__name__)

# Paths are anchored to the project folder (the parent of src/), so the script
# works no matter which folder it is launched from.
def _find_root() -> Path:
    """Return the project folder (the one containing src/ and data/).

    Normally this is the parent of this file's folder. If the code is pasted
    into a notebook (where __file__ does not exist), fall back to the
    CALIFORNIA_HOUSING_ROOT environment variable, a PROJECT_ROOT variable
    defined in the notebook, or the current folder / its parents.
    """
    try:
        return Path(__file__).resolve().parents[1]
    except NameError:
        pass
    import os
    env = os.environ.get("CALIFORNIA_HOUSING_ROOT") or str(globals().get("PROJECT_ROOT", "") or "")
    if env:
        return Path(env).expanduser().resolve()
    here = Path.cwd().resolve()
    for folder in [here, *here.parents]:
        if (folder / "src" / "data_prep.py").exists():
            return folder
    # Last resort: search the home folder for a project with src/data_prep.py
    skip = {"Library", "anaconda3", "miniconda3", "site-packages", "node_modules",
            "Applications", "Pictures", "Music", "Movies", "venv", ".venv"}
    hits = []
    for root, dirs, files in os.walk(Path.home()):
        dirs[:] = [d for d in dirs if not d.startswith(".") and d not in skip]
        if Path(root).name == "src" and "data_prep.py" in files and "model.py" in files:
            hits.append(Path(root).parent.resolve())
```

```
    if len(hits) == 1:
        return hits[0]
    raise FileNotFoundError(
        "Could not locate the project folder (the one containing src/ and data/). "
        "Set PROJECT_ROOT = '<your actual project folder path>' before running, "
        "or run the script from a terminal: python3 src/data_prep.py"
        + (f" (found several candidates: {hits})" if hits else ""))


ROOT = _find_root()
RAW_DATA_PATH = ROOT / "data" / "housing.csv"
CLEAN_DATA_PATH = ROOT / "data" / "housing_clean.csv"

TOP_CODE_VALUE = 500001.0  # the source data caps median_house_value here
IQR_COLUMNS = ["rooms_per_household", "population_per_household"]


def load_raw_data(path: Path) -> pd.DataFrame:
    """Load the raw housing CSV file into a DataFrame."""
    logger.info("Loading raw data from %s", path)
    df = pd.read_csv(path)
    logger.info("Loaded %d rows and %d columns", *df.shape)
    return df


def report_missing_values(df: pd.DataFrame) -> pd.Series:
    """Return a Series of missing-value counts per column, descending."""
    missing = df.isna().sum().sort_values(ascending=False)
    missing = missing[missing > 0]
    logger.info("Missing values by column:\n%s", missing.to_string())
    return missing


def drop_duplicates(df: pd.DataFrame) -> pd.DataFrame:
    """Remove exact duplicate rows."""
    n_before = len(df)
    df = df.drop_duplicates()
    n_removed = n_before - len(df)
    logger.info("Removed %d duplicate rows", n_removed)
    return df
```

```python
def remove_invalid_records(df: pd.DataFrame) -> pd.DataFrame:
    """Remove rows with non-physical or out-of-range values."""
    (parameter) df: Any | DataFrame
    df = df[df["median_house_value"] > 0]
    df = df[df["housing_median_age"] > 0]
    df = df[df["total_rooms"] > 0]
    df = df[df["households"] > 0]
    # Bedrooms cannot logically exceed total rooms.
    df = df[(df["total_bedrooms"].isna()) | (df["total_bedrooms"] <= df["total_rooms"])]
    n_removed = n_before - len(df)
    logger.info("Removed %d invalid/non-physical rows", n_removed)
    return df


def cap_top_coded_target(df: pd.DataFrame, cap_value: float = TOP_CODE_VALUE) -> pd.DataFrame:
    """Remove records at the known top-coded value of the target.

    The median_house_value variable is capped at $500,001 in the source
    data, which creates an artificial spike and would bias a regression
    model. We drop these top-coded rows, so the model's scope is block
    groups with a median value below $500,000.
    """
    n_before = len(df)
    df = df[df["median_house_value"] < cap_value]
    n_removed = n_before - len(df)
    logger.info("Removed %d top-coded records at/above $%.0f", n_removed, cap_value)
    return df


def impute_missing_bedrooms(df: pd.DataFrame) -> pd.DataFrame:
    """Impute missing total_bedrooms using the median within each
    ocean_proximity group, falling back to the global median.
    """
    df = df.copy()
    group_median = df.groupby("ocean_proximity")["total_bedrooms"].transform("median")
    df["total_bedrooms"] = df["total_bedrooms"].fillna(group_median)
    df["total_bedrooms"] = df["total_bedrooms"].fillna(df["total_bedrooms"].median())
    # An imputed value can occasionally exceed that row's own total_rooms
    # when the row has an unusually small room count; cap it so the
    # derived bedrooms_per_room ratio stays logically valid (<= 1).
    df["total_bedrooms"] = np.minimum(df["total_bedrooms"], df["total_rooms"])
    return df
```

```python
def engineer_features(df: pd.DataFrame) -> pd.DataFrame:
    """Create derived ratio features that are more informative than
    the raw counts alone.
    """
    df = df.copy()
    df["rooms_per_household"] = df["total_rooms"] / df["households"]
    df["bedrooms_per_room"] = df["total_bedrooms"] / df["total_rooms"]
    df["population_per_household"] = df["population"] / df["households"]
    return df


def remove_outliers_iqr(df: pd.DataFrame, columns: list) -> pd.DataFrame:
    """Remove rows where any listed column falls outside the Tukey fences
    (Q1 - 1.5*IQR, Q3 + 1.5*IQR).

    Note: the quartiles are computed on the full dataset before the
    train/test split. A leakage-free variant (bounds fitted on the training
    set only) gave essentially the same results in our sensitivity check.
    """
    df = df.copy()
    mask = pd.Series(True, index=df.index)
    for col in columns:
        q1, q3 = df[col].quantile([0.25, 0.75])
        iqr = q3 - q1
        lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
        mask &= df[col].between(lower, upper)
    n_removed = (~mask).sum()
    logger.info("Removed %d outlier rows based on IQR rule for %s", n_removed, columns)
    return df[mask]


def clean_pipeline(raw_path: Path = RAW_DATA_PATH) -> pd.DataFrame:
    """Run the full cleaning pipeline and return the cleaned DataFrame."""
    df = load_raw_data(raw_path)
    report_missing_values(df)  # logged for the record; no return value needed
    df = drop_duplicates(df)
    df = remove_invalid_records(df)
    df = cap_top_coded_target(df)
    df = impute_missing_bedrooms(df)
    df = engineer_features(df)
    df = remove_outliers_iqr(df, columns=IQR_COLUMNS)
    df = df.reset_index(drop=True)
```

```
    _sanity_check(df)
    logger.info("Final cleaned shape: %d rows, %d columns", *df.shape)
    return df

Click to add a breakpoint
def _sanity_check(df: pd.DataFrame) -> None:
    """Fail loudly if the cleaned data violates basic expectations."""
    assert df.isna().sum().sum() == 0, "Cleaned data still has missing values"
    assert (df["median_house_value"] < TOP_CODE_VALUE).all(), "Top-coded rows remain"
    assert (df["bedrooms_per_room"] <= 1).all(), "bedrooms_per_room above 1"
    assert not df.duplicated().any(), "Duplicate rows remain"


def main() -> None:
    """Entry point: run the cleaning pipeline and save the output."""
    clean_df = clean_pipeline()
    CLEAN_DATA_PATH.parent.mkdir(parents=True, exist_ok=True)
    clean_df.to_csv(CLEAN_DATA_PATH, index=False)
    logger.info("Saved cleaned data to %s", CLEAN_DATA_PATH)


if __name__ == "__main__":
    main()
```

```
INFO: Loading raw data from /Users/francoislubin/Downloads/california-housing-project/data/housing.csv
INFO: Loaded 20640 rows and 10 columns
INFO: Missing values by column:
total_bedrooms    207
INFO: Removed 0 duplicate rows
INFO: Removed 0 invalid/non-physical rows
INFO: Removed 965 top-coded records at/above $500001
INFO: Removed 1100 outlier rows based on IQR rule for ['rooms_per_household', 'population_per_household']
INFO: Final cleaned shape: 18575 rows, 13 columns
INFO: Saved cleaned data to /Users/francoislubin/Downloads/california-housing-project/data/housing_clean.csv
```

INFO: Loading raw data from /Users/francoislubin/Downloads/california-housing-project/data/housing.csv
INFO: Loaded 20640 rows and 10 columns
INFO: Missing values by column:
total_bedrooms   207
INFO: Removed 0 duplicate rows
INFO: Removed 0 invalid/non-physical rows
INFO: Removed 965 top-coded records at/above $500001
INFO: Removed 1100 outlier rows based on IQR rule for ['rooms_per_household', 'population_per_household']
INFO: Final cleaned shape: 18575 rows, 13 columns
INFO: Saved cleaned data to /Users/francoislubin/Downloads/california-housing-project/data/housing_clean.csv

Output of: python3 src/model.py

```
"""Model development and evaluation for the California Housing project.

Trains and compares a linear regression baseline against a random
forest regressor using a scikit-learn preprocessing pipeline, k-fold
cross-validation, and held-out test-set evaluation. Saves the selected
final model and a metrics summary.
"""

import json
import logging
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
from scipy import stats
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestRegressor
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

logging.basicConfig(level=logging.INFO, format="%(levelname)s: %(message)s")
logger = logging.getLogger(__name__)

# Paths are anchored to the project folder (the parent of src/), so the script
# works no matter which folder it is launched from.
def _find_root() -> Path:
    """Return the project folder (the one containing src/ and data/).

    Normally this is the parent of this file's folder. If the code is pasted
    into a notebook (where __file__ does not exist), fall back to the
    CALIFORNIA_HOUSING_ROOT environment variable, a PROJECT_ROOT variable
    defined in the notebook, or the current folder / its parents.
    """
    try:
        return Path(__file__).resolve().parents[1]
    except NameError:
```

```
        pass
    import os
    env = os.environ.get("CALIFORNIA_HOUSING_ROOT") or str(globals().get("PROJECT_ROOT", "") or "")
    if env:
        return Path(env).expanduser().resolve()
    here = Path.cwd().resolve()
    for folder in [here, *here.parents]:
        if (folder / "src" / "data_prep.py").exists():
            return folder
    # Last resort: search the home folder for a project with src/data_prep.py
    skip = {"Library", "anaconda3", "miniconda3", "site-packages", "node_modules",
            "Applications", "Pictures", "Music", "Movies", "venv", ".venv"}
    hits = []
    for root, dirs, files in os.walk(Path.home()):
        dirs[:] = [d for d in dirs if not d.startswith(".") and d not in skip]
        if Path(root).name == "src" and "data_prep.py" in files and "model.py" in files:
            hits.append(Path(root).parent.resolve())
    if len(hits) == 1:
        return hits[0]
    raise FileNotFoundError(
        "Could not locate the project folder (the one containing src/ and data/). "
        "Set PROJECT_ROOT = '<your actual project folder path>' before running, "
        "or run the script from a terminal: python3 src/data_prep.py"
        + (f" (found several candidates: {hits})" if hits else ""))


ROOT = _find_root()
DATA_PATH = ROOT / "data" / "housing_clean.csv"
MODELS_DIR = ROOT / "models"
REPORTS_DIR = ROOT / "reports"

TARGET = "median_house_value"
NUMERIC_FEATURES = [
    "longitude",
    "latitude",
    "housing_median_age",
    "total_rooms",
    "total_bedrooms",
    "population",
    "households",
    "median_income",
    "rooms_per_household",
    "bedrooms_per_room",
```

```
    "population_per_household",
]
CATEGORICAL_FEATURES = ["ocean_proximity"]
RANDOM_STATE = 42


def load_data(path: Path = DATA_PATH) -> pd.DataFrame:
    """Load the cleaned dataset."""
    df = pd.read_csv(path)
    logger.info("Loaded data for modeling: %d rows", len(df))
    return df


def build_preprocessor() -> ColumnTransformer:
    """Construct the preprocessing pipeline for numeric and categorical
    features: median imputation + standardization for numeric columns,
    one-hot encoding for the categorical column.
    """
    numeric_pipeline = Pipeline(steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ])
    categorical_pipeline = Pipeline(steps=[
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ])
    preprocessor = ColumnTransformer(transformers=[
        ("num", numeric_pipeline, NUMERIC_FEATURES),
        ("cat", categorical_pipeline, CATEGORICAL_FEATURES),
    ])
    return preprocessor


def evaluate_model(model: Pipeline, x_test: pd.DataFrame, y_test: pd.Series) -> dict:
    """Compute standard regression metrics (RMSE, MAE, R^2) on a data set.

    Used for both the held-out test set and, to check for overfitting,
    the training set.
    """
    y_pred = model.predict(x_test)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
    return {"rmse": rmse, "mae": mae, "r2": r2}
```

```python
def cross_validate_model(
    model: Pipeline, x_train: pd.DataFrame, y_train: pd.Series, cv: int = 5
) -> dict:
    """Run k-fold cross-validation and return mean, sample SD and per-fold RMSE.

    For regressors, cross_val_score uses unshuffled KFold, so the folds are
    identical for every model and can be compared fold by fold.
    """
    scores = cross_val_score(
        model, x_train, y_train, cv=cv, scoring="neg_root_mean_squared_error"
    )
    rmse_scores = -scores
    return {
        "cv_rmse_mean": rmse_scores.mean(),
        "cv_rmse_std": rmse_scores.std(ddof=1),  # sample SD across the folds
        "cv_rmse_folds": rmse_scores.tolist(),
    }


def residual_normality_test(
    model: Pipeline, x_test: pd.DataFrame, y_test: pd.Series,
    sample_size: int = 5000
) -> dict:
    """Run a Shapiro-Wilk test for normality on (up to sample_size) residuals.

    With fewer than sample_size test rows, all residuals are used.
    """
    residuals = y_test.values - model.predict(x_test)
    rng = np.random.default_rng(RANDOM_STATE)
    sample = rng.choice(
        residuals, size=min(sample_size, len(residuals)), replace=False
    )
    stat, p_value = stats.shapiro(sample)
    return {"shapiro_stat": stat, "shapiro_p": p_value}


def train_and_select_model() -> None:
    """Train baseline and candidate models, select the best on
    cross-validated RMSE, evaluate on the test set, and persist results.
    """
    df = load_data()
    x = df[NUMERIC_FEATURES + CATEGORICAL_FEATURES]
```

```
y = df[TARGET]

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=RANDOM_STATE
)
logger.info("Train size: %d, Test size: %d", len(x_train), len(x_test))

# Each pipeline gets its own preprocessor so the two models never share
# fitted state.
linear_model = Pipeline(steps=[
    ("preprocessor", build_preprocessor()),
    ("regressor", LinearRegression()),
])
forest_model = Pipeline(steps=[
    ("preprocessor", build_preprocessor()),
    ("regressor", RandomForestRegressor(
        n_estimators=150, max_depth=18, min_samples_leaf=3,
        random_state=RANDOM_STATE, n_jobs=-1,
    )),
])

results = {}
candidates = [
    ("linear_regression", linear_model),
    ("random_forest", forest_model),
]
for name, model in candidates:
    logger.info("Cross-validating %s ...", name)
    cv_metrics = cross_validate_model(model, x_train, y_train)
    model.fit(x_train, y_train)
    test_metrics = evaluate_model(model, x_test, y_test)
    # Training-set metrics let readers see the train-vs-test gap directly.
    train_metrics = {
        f"train_{k}": v for k, v in evaluate_model(model, x_train, y_train).items()
    }
    results[name] = {**cv_metrics, **test_metrics, **train_metrics}
    logger.info("%s -> %s", name, results[name])

best_name = min(results, key=lambda k: results[k]["cv_rmse_mean"])
best_model = linear_model if best_name == "linear_regression" else forest_model
logger.info("Selected final model: %s", best_name)
```

```python
    residual_stats = residual_normality_test(best_model, x_test, y_test)
    results[best_name].update(residual_stats)

    MODELS_DIR.mkdir(parents=True, exist_ok=True)
    REPORTS_DIR.mkdir(parents=True, exist_ok=True)
    joblib.dump(best_model, MODELS_DIR / "final_model.joblib", compress=3)

    summary = {"selected_model": best_name, "results": results}
    with open(REPORTS_DIR / "model_metrics.json", "w", encoding="utf-8") as f:
        json.dump(summary, f, indent=2, default=float)
    logger.info("Saved model and metrics summary to %s and %s",
                MODELS_DIR, REPORTS_DIR)

    if best_name == "random_forest":
        importances = best_model.named_steps["regressor"].feature_importances_
        feature_names = (
            NUMERIC_FEATURES
            + list(best_model.named_steps["preprocessor"]
                   .named_transformers_["cat"]
                   .named_steps["onehot"]
                   .get_feature_names_out(CATEGORICAL_FEATURES))
        )
        importance_df = pd.DataFrame({
            "feature": feature_names, "importance": importances
        }).sort_values("importance", ascending=False)
        importance_df.to_csv(REPORTS_DIR / "feature_importance.csv", index=False)
        logger.info("Top features:\n%s", importance_df.head(10).to_string(index=False))


def main() -> None:
    """Entry point: train, evaluate, and save the final model."""
    train_and_select_model()


if __name__ == "__main__":
    main()
```

```
✓ 4.9s                                                                    Python

INFO: Loaded data for modeling: 18575 rows
INFO: Train size: 14860, Test size: 3715
INFO: Cross-validating linear_regression ...
INFO: linear_regression -> {'cv_rmse_mean': np.float64(57477.80084024814), 'cv_rmse_std': np.float64(1208.1145641161158), 'cv_rmse_folds': [57036.34929080748, 56203.24967619639, 56754.42955725
INFO: Cross-validating random_forest ...
INFO: random_forest -> {'cv_rmse_mean': np.float64(45203.5992903974), 'cv_rmse_std': np.float64(1391.495015087088), 'cv_rmse_folds': [44876.67846799184, 43565.04149295588, 45122.723864986394,
INFO: Selected final model: random_forest
INFO: Saved model and metrics summary to /Users/francoislubin/Downloads/California-housing-project-main/models and /Users/francoislubin/Downloads/California-housing-project-main/reports
INFO: Top features:
              feature  importance
        median_income    0.428546
  ocean_proximity_INLAND    0.180714
population_per_household    0.111351
            longitude    0.077372
             latitude    0.069072
     housing_median_age    0.041780
     rooms_per_household    0.024787
      bedrooms_per_room    0.022412
            total_rooms    0.010836
             population    0.009878
```

```
INFO: Loaded data for modeling: 18575 rows
INFO: Train size: 14860, Test size: 3715
INFO: Cross-validating linear_regression ...
INFO: linear_regression -> {'cv_rmse_mean': 57477.80084024812, 'cv_rmse_std': 1208.1145641161102, 'cv_rmse_folds':
    [57036.34929080748, 56203.2496761964, 56754.42955725143, 58187.45632015736, 59207.51935682797], 'rmse':
    58464.119739235415, 'mae': 43136.71415892046, 'r2': 0.6515927047554535, 'train_rmse': 57433.13815154557,
'train_mae':
    42646.60470833941, 'train_r2': 0.6532291965826214}
INFO: Cross-validating random_forest ...
INFO: random_forest -> {'cv_rmse_mean': 45203.5992903974, 'cv_rmse_std': 1391.495015087088, 'cv_rmse_folds':
    [44876.67846799184, 43565.04149295588, 45122.723864986394, 45032.42264792914, 47421.12997812373], 'rmse':
    44715.158571241605, 'mae': 29899.23738753981, 'r2': 0.796193533159184, 'train_rmse': 24831.74643006597,
'train_mae':
    15994.539378451145, 'train_r2': 0.9351765769423134}
INFO: Selected final model: random_forest
INFO: Saved model and metrics summary to <project>/models and <project>/reports
INFO: Top features:
                 feature  importance
           median_income    0.428546
  ocean_proximity_INLAND    0.180714
population_per_household    0.111351
               longitude    0.077372
                latitude    0.069072
      housing_median_age    0.041780
     rooms_per_household    0.024787
       bedrooms_per_room    0.022412
             total_rooms    0.010836
              population    0.009878
```

Output of: python3 src/rigor.py

```
"""Extra rigor checks for the California Housing report.

Run from the project folder:  python3 src/rigor.py   (or paste into a notebook cell)
Imports the team's own src/model.py settings so the setup matches exactly.
"""
import sys
import json
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
from scipy import stats
from sklearn.cluster import KMeans
from sklearn.ensemble import RandomForestRegressor
from sklearn.inspection import permutation_importance
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
from sklearn.model_selection import KFold, GroupKFold, train_test_split
from sklearn.pipeline import Pipeline

warnings.filterwarnings("ignore")
def find_root():
    """Locate the project root (the folder containing src/model.py and data/).
    Order: PROJECT_ROOT variable set in the notebook, CALIFORNIA_HOUSING_ROOT environment
    variable, first command-line / %run argument, the notebook's working folder, this file's folder."""
    import os
    candidates = []
    if globals().get("PROJECT_ROOT"):
        candidates.append(Path(str(PROJECT_ROOT)).expanduser())
    if os.environ.get("CALIFORNIA_HOUSING_ROOT"):
        candidates.append(Path(os.environ["CALIFORNIA_HOUSING_ROOT"]).expanduser())
    if len(sys.argv) > 1 and not sys.argv[1].startswith("-"):
        candidates.append(Path(sys.argv[1]).expanduser())
    candidates.append(Path.cwd())
    try:
        candidates.append(Path(__file__).resolve().parent)
    except NameError:  # running in a notebook: no __file__
        pass
    for start in candidates:
        for folder in [start, *start.resolve().parents]:
```

```
            if (folder / "src" / "model.py").exists():
                return folder.resolve()
    # Last resort: search the home folder for a project that has src/model.py and src/data_prep.py
    print("Searching your home folder for the project (this can take a minute) ...")
    skip = {"Library", "anaconda3", "miniconda3", "site-packages", "node_modules", "Applications",
            "Pictures", "Music", "Movies", "venv", ".venv"}
    hits = []
    for root, dirs, files in os.walk(Path.home()):
        dirs[:] = [d for d in dirs if not d.startswith(".") and d not in skip]
        if Path(root).name == "src" and "model.py" in files and "data_prep.py" in files:
            hits.append(Path(root).parent.resolve())
    if len(hits) == 1:
        print("Found project folder:", hits[0])
        return hits[0]
    if len(hits) > 1:
        raise SystemExit("Found several projects; set PROJECT_ROOT to the one you want:\n  " +
                         "\n  ".join(map(str, hits)))
    looked = "\n  ".join(str(c.resolve()) for c in candidates)
    raise SystemExit("Could not find src/model.py.\nLooked in (and above):\n  " + looked +
                     "\nFix: set PROJECT_ROOT = '/full/path/to/your/project' before this code, "
                     "or run os.chdir('/full/path/to/your/project') first.")


ROOT = find_root()
(ROOT / "reports").mkdir(exist_ok=True)
sys.path.insert(0, str(ROOT / "src"))
import model as M          # team's model.py
import data_prep as D      # team's data_prep.py

RS = M.RANDOM_STATE
out = {}


def rmse(y, p):
    return float(np.sqrt(mean_squared_error(y, p)))


def make_rf():
    return Pipeline([("preprocessor", M.build_preprocessor()),
                     ("regressor", RandomForestRegressor(
                         n_estimators=150, max_depth=18, min_samples_leaf=3,
                         random_state=RS, n_jobs=-1))])
```

```
def make_lr():
    return Pipeline([("preprocessor", M.build_preprocessor()),
                     ("regressor", LinearRegression())])


df = pd.read_csv(ROOT / "data/housing_clean.csv")
X = df[M.NUMERIC_FEATURES + M.CATEGORICAL_FEATURES]
y = df[M.TARGET]
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=RS)

# ---- 1. Train vs test (overfitting) ---------------------------------------
rf, lr = make_rf().fit(Xtr, ytr), make_lr().fit(Xtr, ytr)
res = {}
for name, m in [("linear", lr), ("forest", rf)]:
    ptr, pte = m.predict(Xtr), m.predict(Xte)
    res[name] = {"train_rmse": rmse(ytr, ptr), "test_rmse": rmse(yte, pte),
                 "train_r2": float(r2_score(ytr, ptr)),
                 "test_r2": float(r2_score(yte, pte)),
                 "train_mae": float(mean_absolute_error(ytr, ptr)),
                 "test_mae": float(mean_absolute_error(yte, pte))}
out["train_vs_test"] = res
print("TRAIN vs TEST", json.dumps(res, indent=1))

# ---- 2. Paired per-fold comparison (same folds as cross_val_score) ------
kf = KFold(n_splits=5)  # cross_val_score default for regressors
rows = []
for i, (a, b) in enumerate(kf.split(Xtr)):
    xa, xb, ya, yb = Xtr.iloc[a], Xtr.iloc[b], ytr.iloc[a], ytr.iloc[b]
    r_lr = rmse(yb, make_lr().fit(xa, ya).predict(xb))
    r_rf = rmse(yb, make_rf().fit(xa, ya).predict(xb))
    rows.append((i + 1, r_lr, r_rf, r_lr - r_rf))
fold = pd.DataFrame(rows, columns=["fold", "lr_rmse", "rf_rmse", "diff"])
d = fold["diff"].values
t, p = stats.ttest_rel(fold["lr_rmse"], fold["rf_rmse"])
ci = stats.t.interval(0.95, len(d) - 1, loc=d.mean(), scale=stats.sem(d))
out["paired_folds"] = {"table": fold.round(0).to_dict("records"),
                       "mean_diff": float(d.mean()), "sd_diff": float(d.std(ddof=1)),
                       "ci95": [float(ci[0]), float(ci[1])], "t": float(t), "p": float(p),
                       "rf_better_in_folds": int((d > 0).sum())}
print("PAIRED FOLDS\n", fold.round(0).to_string(index=False))
print("mean diff %.0f  sd %.0f  95%% CI (%.0f, %.0f)  t=%.2f p=%.4g" %
      (d.mean(), d.std(ddof=1), ci[0], ci[1], t, p))
```

```
# ---- 3. Bootstrap CIs on test metrics (RF) ------------------------------
rng = np.random.default_rng(RS)
pte = rf.predict(Xte)
yt = yte.values
n = len(yt)
bs_rmse, bs_r2, bs_mae = [], [], []
for _ in range(2000):
    idx = rng.integers(0, n, n)
    bs_rmse.append(rmse(yt[idx], pte[idx]))
    bs_r2.append(r2_score(yt[idx], pte[idx]))
    bs_mae.append(mean_absolute_error(yt[idx], pte[idx]))
pc = lambda v: [float(np.percentile(v, 2.5)), float(np.percentile(v, 97.5))]
out["bootstrap_rf_test"] = {"rmse": pc(bs_rmse), "r2": pc(bs_r2), "mae": pc(bs_mae)}
print("BOOTSTRAP 95% CI RF test: RMSE", pc(bs_rmse), "R2", pc(bs_r2), "MAE", pc(bs_mae))

# ---- 4. VIF + standardized linear coefficients --------------------------
num = df[M.NUMERIC_FEATURES]
Z = (num - num.mean()) / num.std()
vif = {}
for c in Z.columns:
    others = Z.drop(columns=c)
    r2 = LinearRegression().fit(others, Z[c]).score(others, Z[c])
    vif[c] = float(1 / (1 - r2))
out["vif"] = dict(sorted(vif.items(), key=lambda kv: -kv[1]))
print("VIF", json.dumps(out["vif"], indent=1))
coef = pd.Series(lr.named_steps["regressor"].coef_,
                 index=M.NUMERIC_FEATURES + list(
                     lr.named_steps["preprocessor"].named_transformers_["cat"]
                     .named_steps["onehot"].get_feature_names_out(M.CATEGORICAL_FEATURES)))
out["lr_coef"] = coef.reindex(coef.abs().sort_values(ascending=False).index).round(0).head(10).to_dict()
print("LR COEF (numeric = per 1 SD)\n", pd.Series(out["lr_coef"]).to_string())

# ---- 5. Permutation importance (test set) -------------------------------
pi = permutation_importance(rf, Xte, yte, n_repeats=5, random_state=RS,
                            scoring="neg_root_mean_squared_error", n_jobs=-1)
pim = pd.Series(pi.importances_mean, index=Xte.columns).sort_values(ascending=False)
out["perm_importance_rmse_increase"] = pim.round(0).to_dict()
print("PERMUTATION IMPORTANCE (RMSE increase, $)\n", pim.round(0).to_string())

# ---- 6. Error by price segment and by ocean proximity -------------------
seg = pd.DataFrame({"y": yt, "p": pte, "ocean": Xte["ocean_proximity"].values})
seg["bin"] = pd.cut(seg["y"], [0, 100e3, 200e3, 300e3, 400e3, 500e3])
```

```
    def summ(g):
        return pd.Series({"n": len(g), "rmse": rmse(g.y, g.p),
                          "mae": mean_absolute_error(g.y, g.p),
                          "bias_mean_pred_minus_actual": float((g.p - g.y).mean())})
    by_bin = seg.groupby("bin", observed=True).apply(summ).round(0)
    by_oc = seg.groupby("ocean").apply(summ).round(0)
    out["error_by_value_bin"] = by_bin.reset_index().astype(str).to_dict("records")
    out["error_by_ocean"] = by_oc.reset_index().to_dict("records")
    print("ERROR BY VALUE BIN\n", by_bin.to_string())
    print("ERROR BY OCEAN\n", by_oc.to_string())
    print("ISLAND rows in cleaned data:", int((df.ocean_proximity == "ISLAND").sum()))

    # ---- 7. Spatial blocks vs random CV (RF, whole cleaned data) ------------
    km = KMeans(n_clusters=25, random_state=RS, n_init=10).fit(df[["longitude", "latitude"]])
    groups = km.labels_
    sp, rd = [], []
    for (a, b) in GroupKFold(n_splits=5).split(X, y, groups):
        sp.append(rmse(y.iloc[b], make_rf().fit(X.iloc[a], y.iloc[a]).predict(X.iloc[b])))
    for (a, b) in KFold(n_splits=5, shuffle=True, random_state=RS).split(X):
        rd.append(rmse(y.iloc[b], make_rf().fit(X.iloc[a], y.iloc[a]).predict(X.iloc[b])))
    out["spatial_cv"] = {"random_kfold_rmse_mean": float(np.mean(rd)), "random_sd": float(np.std(rd, ddof=1)),
                         "spatial_groupkfold_rmse_mean": float(np.mean(sp)), "spatial_sd": float(np.std(sp, ddof=1)),
                         "spatial_folds": [float(v) for v in sp]}
    print("SPATIAL CV  random %.0f ± %.0f   spatial-block %.0f ± %.0f" %
          (np.mean(rd), np.std(rd, ddof=1), np.mean(sp), np.std(sp, ddof=1)))

    json.dump(out, open(ROOT / "reports" / "rigor_checks.json", "w"), indent=1, default=float)
    print("saved")

 ✓  13.2s
```

```
TRAIN vs TEST {
 "linear": {
  "train_rmse": 57433.13815154557,
  "test_rmse": 58464.11973923542,
  "train_r2": 0.6532291965826214,
  "test_r2": 0.6515927047554535,
  "train_mae": 42646.60470833941,
  "test_mae": 43136.71415892047
 },
 "forest": {
  "train_rmse": 24831.74643006597,
  "test_rmse": 44715.158571241605,
```

```
   "train_r2": 0.9351765769423134,
   "test_r2": 0.796193533159184,
   "train_mae": 15994.539378451145,
   "test_mae": 29899.237387539808
  }
 }
 PAIRED FOLDS
   fold  lr_rmse  rf_rmse    diff
     1  57036.0  44877.0 12160.0
     2  56203.0  43565.0 12638.0
     3  56754.0  45123.0 11632.0
     4  58187.0  45032.0 13155.0
     5  59208.0  47421.0 11786.0
 ...
 NEAR OCEAN   481.0  56145.0  38592.0                      -5101.0
 ISLAND rows in cleaned data: 5
 SPATIAL CV  random 44347 ± 1393   spatial-block 59977 ± 7806
 saved
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```
TRAIN vs TEST {
 "linear": {
  "train_rmse": 57433.13815154557,
  "test_rmse": 58464.119739235415,
  "train_r2": 0.6532291965826214,
  "test_r2": 0.6515927047554535,
  "train_mae": 42646.60470833941,
  "test_mae": 43136.71415892046
 },
 "forest": {
  "train_rmse": 24831.746430065967,
  "test_rmse": 44715.158571241605,
  "train_r2": 0.9351765769423134,
  "test_r2": 0.796193533159184,
  "train_mae": 15994.539378451145,
  "test_mae": 29899.237387539808
 }
}
PAIRED FOLDS
  fold  lr_rmse  rf_rmse    diff
    1  57036.0  44877.0 12160.0
    2  56203.0  43565.0 12638.0
    3  56754.0  45123.0 11632.0
    4  58187.0  45032.0 13155.0
    5  59208.0  47421.0 11786.0
mean diff 12274  sd 627  95% CI (11496, 13053)  t=43.78 p=1.628e-06
BOOTSTRAP 95% CI RF test: RMSE [42752.92847772888, 46809.21732547043] R2 [0.7784668719185027, 0.8117285554015089] MAE
    [28820.516804447503, 31018.86124838968]
VIF {
 "households": 92.85565163189489,
 "total_bedrooms": 65.9309291898919,
 "population": 26.14733758296487,
 "total_rooms": 23.898151440428013,
 "latitude": 10.210663845143738,
 "longitude": 9.66174873316559,
 "rooms_per_household": 5.929642391047449,
 "bedrooms_per_room": 4.222679159521616,
 "population_per_household": 3.069234611185472,
 "median_income": 2.535218101923161,
 "housing_median_age": 1.3069316780797693
}
LR COEF (numeric = per 1 SD)
 ocean_proximity_ISLAND        122803.0
median_income                  60845.0
latitude                      -57227.0
longitude                     -54938.0
ocean_proximity_INLAND        -51668.0
ocean_proximity_NEAR BAY      -30891.0
households                     27740.0
ocean_proximity_NEAR OCEAN    -22093.0
population_per_household      -22068.0
```

```
ocean_proximity_<1H OCEAN     -18151.0
PERMUTATION IMPORTANCE (RMSE increase, $)
 median_income               42430.0
latitude                     20576.0
ocean_proximity              19897.0
longitude                    19357.0
population_per_household     17601.0
housing_median_age            6562.0
rooms_per_household           1172.0
bedrooms_per_room              716.0
total_rooms                    294.0
total_bedrooms                 250.0
households                     129.0
population                     111.0
ERROR BY VALUE BIN
                            n       rmse      mae  bias_mean_pred_minus_actual
bin
(0.0, 100000.0]         702.0   28077.0  17926.0                      15287.0
(100000.0, 200000.0]   1484.0   34988.0  23896.0                      13994.0
(200000.0, 300000.0]    967.0   42086.0  31622.0                       -325.0
(300000.0, 400000.0]    407.0   56191.0  42753.0                     -25724.0
(400000.0, 500000.0]    155.0  115286.0  97102.0                     -96630.0
ERROR BY OCEAN
                  n     rmse      mae  bias_mean_pred_minus_actual
ocean
<1H OCEAN    1622.0  45384.0  31858.0                       3142.0
INLAND       1195.0  34322.0  21200.0                       1903.0
NEAR BAY      417.0  52813.0  37184.0                       1968.0
NEAR OCEAN    481.0  56145.0  38592.0                      -5101.0
ISLAND rows in cleaned data: 5
SPATIAL CV  random 44347 ± 1393   spatial-block 59977 ± 7806
saved
```

Output of: python3 src/sensitivity.py

```python
"""Cleaning-choice sensitivity and leakage-free variant. Run from the project folder:  python3 src/sensitivity.py   (or paste into a notebook cell)"""
import sys
import json
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline

warnings.filterwarnings("ignore")
def find_root():
    """Locate the project root (the folder containing src/model.py and data/).
    Order: PROJECT_ROOT variable set in the notebook, CALIFORNIA_HOUSING_ROOT environment
    variable, first command-line / %run argument, the notebook's working folder, this file's folder."""
    import os
    candidates = []
    if globals().get("PROJECT_ROOT"):
        candidates.append(Path(str(PROJECT_ROOT)).expanduser())
    if os.environ.get("CALIFORNIA_HOUSING_ROOT"):
        candidates.append(Path(os.environ["CALIFORNIA_HOUSING_ROOT"]).expanduser())
    if len(sys.argv) > 1 and not sys.argv[1].startswith("-"):
        candidates.append(Path(sys.argv[1]).expanduser())
    candidates.append(Path.cwd())
    try:
        candidates.append(Path(__file__).resolve().parent)
    except NameError:  # running in a notebook: no __file__
        pass
    for start in candidates:
        for folder in [start, *start.resolve().parents]:
            if (folder / "src" / "model.py").exists():
                return folder.resolve()
    # Last resort: search the home folder for a project that has src/model.py and src/data_prep.py
    print("Searching your home folder for the project (this can take a minute) ...")
    skip = {"Library", "anaconda3", "miniconda3", "site-packages", "node_modules", "Applications",
            "Pictures", "Music", "Movies", "venv", ".venv"}
```

```
    hits = []
    for root, dirs, files in os.walk(Path.home()):
        dirs[:] = [d for d in dirs if not d.startswith(".") and d not in skip]
        if Path(root).name == "src" and "model.py" in files and "data_prep.py" in files:
            hits.append(Path(root).parent.resolve())
    if len(hits) == 1:
        print("Found project folder:", hits[0])
        return hits[0]
    if len(hits) > 1:
        raise SystemExit("Found several projects; set PROJECT_ROOT to the one you want:\n  " +
                         "\n  ".join(map(str, hits)))
    looked = "\n  ".join(str(c.resolve()) for c in candidates)
    raise SystemExit("Could not find src/model.py.\nLooked in (and above):\n  " + looked +
                     "\nFix: set PROJECT_ROOT = '/full/path/to/your/project' before this code, "
                     "or run os.chdir('/full/path/to/your/project') first.")


ROOT = find_root()
(ROOT / "reports").mkdir(exist_ok=True)
sys.path.insert(0, str(ROOT / "src"))
import model as M
import data_prep as D

RS = M.RANDOM_STATE
FEATS = M.NUMERIC_FEATURES + M.CATEGORICAL_FEATURES
IQR_COLS = ["rooms_per_household", "population_per_household"]


def rf():
    return Pipeline([("preprocessor", M.build_preprocessor()),
                     ("regressor", RandomForestRegressor(
                         n_estimators=150, max_depth=18, min_samples_leaf=3,
                         random_state=RS, n_jobs=-1))])


def lr():
    return Pipeline([("preprocessor", M.build_preprocessor()),
                     ("regressor", LinearRegression())])


def evaluate(df_train, df_test, label, extra_mask_test=None):
    row = {"variant": label, "n_train": len(df_train), "n_test": len(df_test)}
```

```
    for name, mk in [("rf", rf), ("lr", lr)]:
        m = mk().fit(df_train[FEATS], df_train[M.TARGET])
        p = m.predict(df_test[FEATS])
        y = df_test[M.TARGET].values
        row[f"{name}_rmse"] = float(np.sqrt(mean_squared_error(y, p)))
        row[f"{name}_r2"] = float(r2_score(y, p))
        if extra_mask_test is not None:
            mk_ = extra_mask_test(df_test).values
            row[f"{name}_rmse_under500k"] = float(np.sqrt(mean_squared_error(y[mk_], p[mk_])))
    return row


def pipeline_variant(topcode=True, iqr=True):
    df = D.load_raw_data(ROOT / "data/housing.csv")
    df = D.drop_duplicates(df)
    df = D.remove_invalid_records(df)
    if topcode:
        df = D.cap_top_coded_target(df)
    df = D.impute_missing_bedrooms(df)
    df = D.engineer_features(df)
    if iqr:
        df = D.remove_outliers_iqr(df, IQR_COLS)
    return df.reset_index(drop=True)


def split(df):
    return train_test_split(df, test_size=0.2, random_state=RS)


rows = []
# 1 baseline (team pipeline)
tr, te = split(pipeline_variant())
rows.append(evaluate(tr, te, "baseline (team pipeline)"))
# 2 no IQR removal
tr, te = split(pipeline_variant(iqr=False))
rows.append(evaluate(tr, te, "no IQR outlier removal"))
# 3 keep capped rows (evaluate on all and on <500k only)
tr, te = split(pipeline_variant(topcode=False))
rows.append(evaluate(tr, te, "keep $500,001 capped rows",
                     extra_mask_test=lambda d: d[M.TARGET] < 500001))
```

```
    # 4 leakage-free: row-level rules -> split -> train-only imputation + IQR bounds
    raw = D.load_raw_data(ROOT / "data/housing.csv")
    raw = D.drop_duplicates(raw)
    raw = D.remove_invalid_records(raw)
    raw = D.cap_top_coded_target(raw)
    tr, te = train_test_split(raw, test_size=0.2, random_state=RS)
    tr, te = tr.copy(), te.copy()
    med_by_group = tr.groupby("ocean_proximity")["total_bedrooms"].median()
    glob_med = tr["total_bedrooms"].median()
    for d in (tr, te):
        d["total_bedrooms"] = d["total_bedrooms"].fillna(d["ocean_proximity"].map(med_by_group))
        d["total_bedrooms"] = d["total_bedrooms"].fillna(glob_med)
        d["total_bedrooms"] = np.minimum(d["total_bedrooms"], d["total_rooms"])
    tr, te = D.engineer_features(tr), D.engineer_features(te)
    mask_tr, mask_te = pd.Series(True, index=tr.index), pd.Series(True, index=te.index)
    for c in IQR_COLS:
        q1, q3 = tr[c].quantile([0.25, 0.75])
        lo, hi = q1 - 1.5 * (q3 - q1), q3 + 1.5 * (q3 - q1)
        mask_tr &= tr[c].between(lo, hi)
        mask_te &= te[c].between(lo, hi)
    rows.append(evaluate(tr[mask_tr], te[mask_te], "leakage-free (split first; train-only medians & IQR bounds)"))
    # 4b same, but test set keeps ALL rows (no outlier filter on test)
    rows.append(evaluate(tr[mask_tr], te, "leakage-free, test keeps outliers"))

    res = pd.DataFrame(rows)
    pd.set_option("display.width", 250)
    print(res.round(3).to_string(index=False))
    json.dump(rows, open(ROOT / "reports" / "sensitivity.json", "w"), indent=1)

✓ 4.5s
```

```
INFO: Loading raw data from /Users/francoislubin/Downloads/California-housing-project-main/data/housing.csv
INFO: Loaded 20640 rows and 10 columns
INFO: Removed 0 duplicate rows
INFO: Removed 0 invalid/non-physical rows
INFO: Removed 965 top-coded records at/above $500001
INFO: Removed 1100 outlier rows based on IQR rule for ['rooms_per_household', 'population_per_household']
INFO: Loading raw data from /Users/francoislubin/Downloads/California-housing-project-main/data/housing.csv
INFO: Loaded 20640 rows and 10 columns
INFO: Removed 0 duplicate rows
INFO: Removed 0 invalid/non-physical rows
INFO: Removed 965 top-coded records at/above $500001
INFO: Loading raw data from /Users/francoislubin/Downloads/California-housing-project-main/data/housing.csv
```

```
INFO: Loading raw data from /Users/francoislubin/Downloads/California-housing-project-main/data/housing.csv
INFO: Loaded 20640 rows and 10 columns
INFO: Removed 0 duplicate rows
INFO: Removed 0 invalid/non-physical rows
INFO: Removed 1210 outlier rows based on IQR rule for ['rooms_per_household', 'population_per_household']
INFO: Loading raw data from /Users/francoislubin/Downloads/California-housing-project-main/data/housing.csv
INFO: Loaded 20640 rows and 10 columns
INFO: Removed 0 duplicate rows
INFO: Removed 0 invalid/non-physical rows
INFO: Removed 965 top-coded records at/above $500001
                                      variant  n_train  n_test   rf_rmse  rf_r2   lr_rmse  lr_r2  rf_rmse_under500k  lr_rmse_under500k
                     baseline (team pipeline)    14860    3715 44715.159  0.796 58464.120  0.652                NaN                NaN
                       no IQR outlier removal    15740    3935 46362.574  0.784 62105.213  0.613                NaN                NaN
                    keep $500,001 capped rows    15544    3886 49500.614  0.819 64901.446  0.688          44990.176          57384.901
leakage-free (split first; train-only medians & IQR bounds)    14868    3711 46331.803  0.785 59520.567  0.645                NaN                NaN
                 leakage-free, test keeps outliers    14868    3935 46388.562  0.784 62177.376  0.612                NaN                NaN
```

```
INFO: Loading raw data from /Users/francoislubin/Downloads/california-housing-project-main/data/housing.csv
INFO: Loaded 20640 rows and 10 columns
INFO: Removed 0 duplicate rows
INFO: Removed 0 invalid/non-physical rows
INFO: Removed 965 top-coded records at/above $500001
INFO: Removed 1100 outlier rows based on IQR rule for ['rooms_per_household', 'population_per_household']
INFO: Loading raw data from /Users/francoislubin/Downloads/california-housing-project-main/data/housing.csv
INFO: Loaded 20640 rows and 10 columns
INFO: Removed 0 duplicate rows
INFO: Removed 0 invalid/non-physical rows
```

```
INFO: Removed 965 top-coded records at/above $500001
INFO: Loading raw data from /Users/francoislubin/Downloads/california-housing-project-main/data/housing.csv
INFO: Loaded 20640 rows and 10 columns
INFO: Removed 0 duplicate rows
INFO: Removed 0 invalid/non-physical rows
INFO: Removed 1210 outlier rows based on IQR rule for ['rooms_per_household', 'population_per_household']
INFO: Loading raw data from /Users/francoislubin/Downloads/california-housing-project-main/data/housing.csv
INFO: Loaded 20640 rows and 10 columns
INFO: Removed 0 duplicate rows
INFO: Removed 0 invalid/non-physical rows
INFO: Removed 965 top-coded records at/above $500001
                                                variant  n_train  n_test   rf_rmse  rf_r2   lr_rmse  lr_r2
    rf_rmse_under500k  lr_rmse_under500k
                                  baseline (team pipeline)    14860    3715 44715.159  0.796 58464.120  0.652
                   NaN                NaN
                                    no IQR outlier removal    15740    3935 46362.574  0.784 62105.213  0.613
                   NaN                NaN
                                 keep $500,001 capped rows    15544    3886 49500.614  0.819 64901.446  0.688
    44990.176          57384.901
leakage-free (split first; train-only medians & IQR bounds)    14868    3711 46331.803  0.785 59520.567  0.645
                   NaN                NaN
                         leakage-free, test keeps outliers    14868    3935 46388.562  0.784 62177.376  0.612
                   NaN                NaN
```

Output of: python3 src/hyperparam_check.py .

```
    """Compare the team's constrained random forest with a larger, unconstrained one."""
    from
    impor
    sys.p
    from sklearn.ensemble import RandomForestRegressor
    from sklearn.pipeline import Pipeline
    from sklearn.model_selection import train_test_split
    from sklearn.metrics import mean_squared_error, r2_score
    import model as M, data_prep as D
    df = pd.read_csv(D.CLEAN_DATA_PATH)
    y = df["median_house_value"]; X = df.drop(columns="median_house_value")
    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=42)
    for name, kw in [("team (150 trees, depth 18, leaf 3)", dict(n_estimators=150, max_depth=18, min_samples_leaf=3)),
                     ("larger (300 trees, unconstrained)", dict(n_estimators=300))]:
        pipe = Pipeline([("prep", M.build_preprocessor()), ("m", RandomForestRegressor(random_state=42, n_jobs=-1, **kw))])
        pipe.fit(Xtr, ytr); p = pipe.predict(Xte)
        buf = io.BytesIO(); joblib.dump(pipe, buf)
        print(f"{name}: RMSE {np.sqrt(mean_squared_error(yte,p)):,.0f}  R2 {r2_score(yte,p):.3f}  size {buf.tell()/1e6:.1f} MB")
```

(module) sklearn

Configure global settings and get information about the working environment.

✓ 2.8s

```
team (150 trees, depth 18, leaf 3): RMSE 44,715  R2 0.796  size 52.1 MB
larger (300 trees, unconstrained): RMSE 44,520  R2 0.798  size 402.3 MB
```

team (150 trees, depth 18, leaf 3): RMSE 44,715 R2 0.796 size 52.1 MB
larger (300 trees, unconstrained): RMSE 44,520 R2 0.798 size 402.3 MB